MINISTRY OF EDUCATION AND TRAINING

UNIVERSITY OF ECONOMICS AND FINANCE

PROJECT REPORT

Information Security for Web Applications

Major: Information Technology

Minor: CyberSecurity

TOPIC: RESEARCH ON METHODS TO DETECT AND PREVENT BOTNET IN WEB SYSTEM

Supervisor: Nguyen Minh Thang, MSc

Student:

Đặng Nguyễn Phi Lót	**ID:225053737**	**22D1ITE-CYB02**
Nguyễn Võ Lâm Nguyên	**ID:225052566**	**22D1ITE-CYB02**
Phạm Phú Quang	**ID:225052134**	**22D1ITE-CYB02**
Nguyễn Nam Huy	**ID:225123259**	**22D1ITE-CYB02**

Ho Chi Minh City, 2026

MINISTRY OF EDUCATION AND TRAINING
UNIVERSITY OF ECONOMICS AND FINANCE

PROJECT REPORT

Information Security for Web Applications

Major: Information Technology

Minor: CyberSecurity

TOPIC: RESEARCH ON METHODS TO DETECT AND PREVENT BOTNET IN WEB SYSTEM

Supervisor: Nguyen Minh Thang, MSc

Student:

Đặng Nguyễn Phi Lót	**ID:225053737**	**22D1ITE-CYB02**
Nguyễn Võ Lâm Nguyên	**ID:225052566**	**22D1ITE-CYB02**
Phạm Phú Quang	**ID:225052134**	**22D1ITE-CYB02**
Nguyễn Nam Huy	**ID:225123259**	**22D1ITE-CYB02**

Ho Chi Minh City, 2026

TABLE OF CONTENTS

Contents

TABLE OF CONTENTS i

LIST OF IMAGES iv

ACKNOWLEDGEMENT v

ABSTRACTS vi

CHAPTER 1 . INTRODUCTION 1

1.1 OVERVIEW OF THE TOPIC 1

1.2 REASONS FOR CHOOSING THE TOPIC 1

1.3 OBJECTIVES OF THE RESEARCH 1

1.4 PROJECT STRUCTURE 2

CHAPTER 2 . THEORETICAL BASIS 3

2.1 Overview of Botnet 3

2.2 Botnet Architecture and Communication Mode 3

2.2.1 Centralized Botnet Architecture 3

2.2.2 Decentralized (Peer-to-Peer) Botnet Architecture 4

2.2.3 Hybrid Botnet Architecture 5

2.2.4 Communication Models in Web-Based Botnets 5

2.2.5 Impact of Architecture on Detection and Prevention 6

2.3 Botnet Activities in Web Systems 6

2.3.1 Credential Theft and Account Takeover 7

2.3.2 Automated HTTP Requests and Resource Abuse 7

2.3.3 Spam Content Generation 8

2.3.4 Brute Force and Password Guessing Attacks 8

2.3.5 Fake Account Creation and Automation Abuse 9
2.3.6 Evasion Techniques Used by Web Botnets 9
2.3.7 Indicators of Botnet Activities in Web Logs 10
2.4 Human vs Bot Behavior Characteristics 10
2.4.1 Request Timing and Frequency 10
2.4.2 Navigation and Interaction Patterns 10
2.4.3 Mouse and Keyboard Dynamics 11
2.4.4 Session Duration and Activity Consistency 11
2.4.5 Error Patterns and Adaptive Behavior 11
2.4.6 Device and Browser Fingerprint Stability 12
2.4.7 Temporal Behavior Patterns 12
2.5 Bot Detection Techniques 12
2.5.1 Behavior-Based Detection 12
2.5.2 CAPTCHA-Based Detection 13
2.5.3 Rate Limiting and Traffic Throttling 13
2.5.4 Browser Fingerprinting 14
2.5.5 Machine Learning-Based Detection 14
2.5.6 IP Reputation and Blacklisting 14
2.5.7 Hybrid Detection Approaches 15
CHAPTER 3 . EXPERIMENTAL RESULTS 16
3.1 Using rule snort 16
3.2 Machine learning 19
3.2.1 Ramdom Forest 19
3.2.2 Machine Learning with Ramdom Forest 20
3.2.3 Testing in Kali 24
CHAPTER 4 . CONCLUSION 27
4.1 CONCLUSION 27

4.2 DEVELOPMENT 27
REFERENCES 29

LIST OF IMAGES

Figure 1: Folder containing rules 17
Figure 2: Run rule 18
Figure 3: Botnet attack 18
Figure 4: Alert system 19
Figure 5: Import Data 20
Figure 6: Label in Dataset 21
Figure 7: Data visualization 21
Figure 8: Heatmap 22
Figure 9: Important labels 23
Figure 10: Result 23
Figure 11: Export files 23
Figure 12: File to detection botnet 24
Figure 13: Code to run script 24
Figure 14: Run script 25
Figure 15: Attack simulator 25
Figure 16: Detection system 26

ACKNOWLEDGEMENT

To successfully complete our capstone project on Research on methods to detect and prevent botnet in web system, our team has invested not only time, effort, and dedication, but also received many support and guidance from our teachers and peers.

First and foremost, we would like to express our sincere gratitude to UEF – University of Economics and Finance for organizing and providing us with the opportunity to study and collaborate during the IIA semester of the academic year 2025–2026. The university has given us access to a meaningful and practical subject, and more importantly, we truly believe in the path we have chosen by entrusting our future to UEF.

We would also like to extend our heartfelt thanks to Mr. Nguyễn Minh Thắng, our instructor. We are truly fortunate to have had him accompany us throughout this journey. His guidance and support have been instrumental in helping us understand the subject matter and complete our project.

This project can be considered a small scientific endeavor. Research on methods to detect and prevent botnet in web system is not easy task for us. Through this topic, we hope our instructor see the effort and commitment reflected in every page of our work. We look forward to receiving thoughtful feedback and constructive suggestions so that we can recognize our shortcomings and gain valuable experience for future assignments and projects.

Once again, we sincerely thank our instructor. We wish him good health and continued success in his teaching career.

ABSTRACTS

Botnet attacks have become one of the most serious threats to modern web systems due to their ability to automate malicious activities at large scale. These attacks commonly include credential theft, brute force login attempts, spam generation, and abnormal automated requests, which can severely affect system availability, data security, and user trust. Traditional security mechanisms based on static rules or simple filtering techniques are often insufficient to detect advanced and adaptive botnets.

This project presents a combined approach for detecting and preventing botnet activities in web systems by integrating a network-based intrusion detection system and a machine learning-based behavior analysis model. Snort is used to monitor network traffic and detect known attack patterns through predefined rules, enabling fast identification of suspicious activities at the network level. In addition, a machine learning model is employed to analyze user behavior at the application level and classify traffic as either human or bot based on behavioral features.

The proposed system is implemented and evaluated on a demo web application with login and comment functionalities. Experimental results demonstrate that the combination of Snort and machine learning improves detection accuracy and reduces false positives compared to using a single detection technique. The layered detection approach provides an effective and flexible solution for protecting web systems against botnet attacks.

This study highlights the importance of integrating signature-based detection with intelligent behavior analysis and provides a practical foundation for developing more advanced botnet defense mechanisms in web environments.

CHAPTER 1 . INTRODUCTION

1.1 OVERVIEW OF THE TOPIC

This project will research what botnets are, how they operate in web systems, and the damage they cause. Through this, we can study their behavior and propose methods for detecting bots.

The goal of the project is to find a method to differentiate between bots and humans, propose anti-bot solutions, and test them on a demo website.

1.2 REASONS FOR CHOOSING THE TOPIC

Botnets are tools used to deliver malware across networks. This defines them as major threats to corporate networks, which may always need data, applications and services available. Botnets are the primary cause of Denial-of-Service (DOS) attacks. They can prevent employees from accessing (sensitive and confidential) information stored within the networks. It may negatively affect the data's availability on the system and the data-handling credibility of the data handler. Despite how common DOS attacks are, businesses may not have any solutions to combat this threat. A classic botnet attack example is the Mirai botnet (2016), which hijacked vulnerable Internet of Things (IoT) devices (cameras, routers) to launch massive Distributed Denial of Service (DDoS) attacks, overwhelming major sites like Netflix and Twitter, and showing how insecure devices can create a massive threat. Other examples include the Emotet botnet, used to spread banking Trojans and ransomware via phishing, and the 3ve botnet, which committed ad fraud by generating fake clicks on ads. The number of botnet attacks is rapidly increasing in today's rapidly developing technological landscape, posing a threat to many businesses. Therefore, this research was conducted to investigate and identify the signs of botnets with the aim of developing preventative methods.

1.3 OBJECTIVES OF THE RESEARCH

Understanding what botnets are, their behavior, and their impact

Detecting and distinguishing botnets from humans

Botnet Detection Demo and provides methods for preventing them.

1.4 PROJECT STRUCTURE

The report has 3 main parts:

Part 1: Introduction

Part 2: Theoretical basis

Part 3: Experiments and results

Part 4: Conclusion

CHAPTER 2 . THEORETICAL BASIS

2.1 OVERVIEW OF BOTNET

A botnet is a network of compromised devices, commonly referred to as bots or zombies, that are remotely controlled by an attacker known as the botmaster. These devices are typically infected with malicious software that allows the attacker to issue commands without the users' knowledge.

In web systems, botnets are widely used to automate malicious activities such as credential theft, spam distribution, brute-force attacks, and large-scale automated requests. Due to their distributed nature, botnets are difficult to detect and mitigate using traditional security mechanisms.

2.2 BOTNET ARCHITECTURE AND COMMUNICATION MODE

Botnet architecture defines how compromised devices (bots) are organized and how they communicate with the botmaster. The communication model directly affects the botnet's scalability, resilience, stealth, and detectability. In web systems, botnets are often designed to blend malicious traffic with legitimate HTTP/HTTPS traffic, making detection significantly more difficult.

2.2.1 Centralized Botnet Architecture

In a centralized botnet architecture, all bots communicate with one or more centralized Command-and-Control (C&C) servers. The botmaster sends commands to the C&C server, which are then forwarded to all connected bots.

Common C&C protocols: Internet Relay Chat (IRC), HTTP/HTTPS, Web-based APIs.

In web-based botnets, HTTP/HTTPS is the most common communication channel because it closely resembles normal web traffic and easily bypasses firewalls.

Operational process:

The bot infects a victim machine.

The bot periodically connects to the C&C server.

The bot receives commands (e.g., attack instructions, target URLs).

The bot executes the commands and may send results back to the server.

Advantages: Simple to implement and manage, fast command distribution and easy to update bot behavior.

Disadvantages: Single point of failure, if the C&C server is detected and taken down, the entire botnet can be disabled.

Relevance to web systems: Centralized botnets are commonly used for web attacks such as brute-force login attempts, spam posting, and automated form submissions due to their simplicity and efficiency.

Centralized botnets are commonly used for web attacks such as brute-force login attempts, spam posting, and automated form submissions due to their simplicity and efficiency.

2.2.2 Decentralized (Peer-to-Peer) Botnet Architecture

In decentralized or peer-to-peer (P2P) botnets, the traditional central command-and-control (C&C) server is eliminated, and instead, bots communicate directly with one another to form a distributed network. Within this structure, each bot is capable of *receiving commands* from its peers, *forwarding* those commands to other bots, and functioning *simultaneously* as both a client and a server. This architecture makes P2P botnets more resilient and difficult to dismantle, since there is no single point of failure that defenders can target to disrupt the entire network.

Communication characteristics: Uses P2P protocols or custom overlays, commands propagate through the botnet network and difficult to identify a single control node.

Advantages: High resilience against takedown, no single point of failure and better survivability under network disruption.

Disadvantages: Complex design and maintenance, slower command propagation, Harder to update bot behavior.

P2P botnets are often used for large-scale attacks such as distributed denial-of-service (DDoS) and long-term spam campaigns. Their distributed nature makes them more difficult to detect using IP-based filtering alone.

2.2.3 Hybrid Botnet Architecture

Hybrid botnet architecture represents a combination of centralized and decentralized models, designed to achieve both effective control and strong resilience. In this structure, central servers are responsible for issuing high-level commands to guide the overall operation of the botnet, while peer-to-peer (P2P) communication is employed to distribute these commands across the network, ensuring scalability and redundancy. Additionally, backup command-and-control (C&C) channels are maintained so that if the primary server is disabled or taken down, the botnet can continue functioning without interruption. This hybrid approach allows attackers to retain centralized oversight while mitigating the risk of a single point of failure, making such botnets more robust and harder to dismantle.

Advantages: Improved fault tolerance, faster command dissemination than pure P2P, Increased stealth and flexibility.

Hybrid botnets are increasingly popular in modern web attacks because they can dynamically switch communication channels and evade detection mechanisms.

2.2.4 Communication Models in Web-Based Botnets

Botnets targeting web systems frequently rely on application-layer protocols to hide malicious activities within legitimate traffic.

-HTTP/HTTPS-Based Communication: Bots send requests to C&C servers using standard HTTP methods (GET, POST). Encrypted HTTPS traffic further complicates inspection and detection.

Benefits for attackers: Traffic blends with normal user behavior, Easily passes through firewalls and proxies, and supports encrypted payloads.

API-Based Communication: Modern botnets often communicate via RESTful APIs hosted on cloud services, compromised websites and content delivery networks (CDNs), so this method increases anonymity and reduces suspicion.

Covert and Obfuscated Channels: Advanced botnets may use: Encrypted payloads, domain Generation Algorithms (DGA) and fast-flux DNS techniques.

These techniques allow bots to dynamically locate new C&C servers, making static blacklisting ineffective.

2.2.5 Impact of Architecture on Detection and Prevention

The architecture of a botnet plays a critical role in shaping both the difficulty of detection and the effectiveness of defense mechanisms. In centralized architectures, detection is moderately challenging since defenders can focus on identifying and disrupting the command-and-control (C&C) server; however, attackers often conceal or rapidly shift these servers to evade takedown. Peer-to-peer (P2P) architectures present a higher level of difficulty because there is no single point of control, making it harder to dismantle the network and requiring defenders to analyze distributed communication patterns. Hybrid architectures pose the greatest challenge, as they combine centralized oversight with decentralized resilience. Their adaptive communication channels allow botnets to continue operating even if one control mechanism is disabled, significantly complicating detection and prevention efforts.

Understanding these architectural differences is essential for designing robust bot detection mechanisms in web systems. Effective strategies often require a layered approach that integrates behavior analysis, rate limiting, machine learning-based classification, and browser fingerprinting. By tailoring defenses to the specific challenges posed by each architecture, web systems can improve their resilience against botnet-driven attacks such as credential theft, automated requests, spam, and brute-force attempts.

2.3 BOTNET ACTIVITIES IN WEB SYSTEMS

Botnets are widely used to perform automated and large-scale malicious activities against web systems. By leveraging a large number of compromised devices, attackers can execute attacks that are difficult to detect and mitigate using traditional security mechanisms. This section describes the most common botnet-driven activities targeting web applications.

2.3.1 Credential Theft and Account Takeover

One of the primary objectives of botnets in web systems is credential theft. Botnets commonly perform *credential stuffing* and *password spraying* attacks using large databases of leaked usernames and passwords obtained from previous data breaches.

Attack process:

-The botnet retrieves credential lists from the C&C server.

-Bots automatically submit login requests to the target website.

-Successful login attempts are reported back to the attacker.

-Compromised accounts are exploited or sold on underground markets.

Impact on web systems: Unauthorized access to user accounts, exposure of personal and financial data, loss of user trust and reputational damage. Botnets are particularly effective in credential theft due to their ability to distribute login attempts across many IP addresses, thereby evading basic rate-limiting and IP-blocking defenses.

2.3.2 Automated HTTP Requests and Resource Abuse

Botnets are capable of generating massive volumes of automated HTTP/HTTPS requests directed at web servers. Although these requests often conform to protocol standards and may appear legitimate, they are typically produced by scripts or automated tools rather than genuine human users. Such activity enables attackers to exploit web resources in several ways, including large-scale web scraping and data harvesting, abuse of public APIs, inventory hoarding for purposes such as ticket or product scalping, and application-layer denial-of-service (Layer 7 DoS) attacks.

Bot-generated traffic exhibits distinctive characteristics that differentiate it from normal user interactions. These include extremely high request frequencies, uniform or repetitive request patterns, very short intervals between successive requests, and minimal or nonexistent client-side interaction (e.g., no mouse movement or keystroke activity). The cumulative effect of these behaviors can

severely degrade server performance, increase operational costs, and reduce service availability for legitimate users, thereby undermining the reliability and security of web systems.

2.3.3 Spam Content Generation

Botnets are widely employed to automate the posting of spam content across web platforms such as blogs, forums, and comment sections. These activities typically involve the dissemination of unsolicited advertising messages, the injection of malicious or phishing links, the creation of fake user-generated content, and the manipulation of online reviews and ratings to distort public perception. By leveraging automation, bots can register large numbers of accounts, bypass weak CAPTCHA mechanisms, and submit spam at a scale and speed far beyond human capability.

The consequences of such spam campaigns are significant. They reduce the overall quality and trustworthiness of online content, undermine the credibility of affected websites, and expose users to heightened risks of malware infection and phishing attacks. As a result, spam content generation not only disrupts user experience but also poses serious security and reputational challenges for web systems.

2.3.4 Brute Force and Password Guessing Attacks

Botnets are often leveraged to conduct brute force attacks, in which they systematically attempt numerous password combinations against authentication interfaces until access is achieved. Typical targets include login pages, administrative panels, and API authentication endpoints, all of which are critical to the security of web systems. Attackers employ techniques such as high-speed password guessing, distributing requests across multiple compromised devices to evade detection, and adaptive retry mechanisms that adjust based on server responses.

Unlike human adversaries, botnets can sustain these attacks continuously without fatigue, making them particularly dangerous for systems with weak

authentication protections. The persistence and scale of brute force campaigns not only threaten account security but also impose significant strain on server resources, underscoring the importance of defenses such as rate limiting, account lockout policies, and multi-factor authentication.

2.3.5 Fake Account Creation and Automation Abuse

Botnets are capable of automatically generating large numbers of fake accounts on web platforms, exploiting weaknesses in registration workflows such as missing email verification or weak CAPTCHA implementations. These fake accounts serve multiple malicious purposes, including amplifying spam and misinformation, bypassing platform usage limits, engaging in fraudulent activities, and conducting social engineering attacks. By automating the account creation process, botnets can overwhelm systems with illegitimate identities at scale, undermining the integrity of online communities, distorting user engagement metrics, and increasing the risk of security breaches.

2.3.6 Evasion Techniques Used by Web Botnets

To evade detection, web botnets employ a variety of sophisticated strategies that enable them to blend in with legitimate traffic. One common technique is IP rotation, achieved through the use of proxies or compromised devices, which helps disguise the origin of requests. Bots also randomize User-Agent strings to mimic different browsers and operating systems, making it harder to identify automated traffic. The use of headless browsers and automation frameworks allows bots to execute complex interactions that resemble genuine user activity. In addition, many botnets simulate human browsing behavior, such as random mouse movements or variable request timing, to avoid behavioral filters. Finally, encrypted communication via HTTPS conceals malicious traffic within normal secure connections, further complicating detection. Collectively, these evasion methods allow botnets to bypass simple rule-based defenses and present themselves as legitimate users, posing significant challenges for web security systems.

2.3.7 Indicators of Botnet Activities in Web Logs

Botnet activity can often be detected through anomalies recorded in web server logs. Typical indicators include unusually high request rates, repeated failed login attempts, identical request patterns across multiple sessions, excessive activity during non-human usage hours, and limited interaction diversity such as the absence of mouse or keyboard events. Careful log analysis is therefore essential for recognizing these patterns and distinguishing automated traffic from legitimate user behavior.

2.4 HUMAN VS BOT BEHAVIOR CHARACTERISTICS

Distinguishing between human users and automated bots is a fundamental challenge in web security. While bots increasingly attempt to mimic human behavior, measurable differences still exist in interaction patterns, timing, and contextual awareness. This section analyzes key behavioral characteristics that can be used to classify web traffic as human or bot.

2.4.1 Request Timing and Frequency

Human users generate web requests at irregular intervals due to natural cognitive and physical constraints, such as reading time and decision-making. In contrast, bots can issue requests at high speed with consistent timing.

Human behavior characteristics: Variable time gaps between requests, pauses for reading or thinking and inconsistent interaction pace.

Bot behavior characteristics: High request frequency, uniform inter-request interval and continuous activity without breaks.

These timing patterns are commonly used as features in rate limiting and behavior-based detection systems.

2.4.2 Navigation and Interaction Patterns

Human users navigate web pages in a logical and context-aware manner. Their actions depend on content, interface design, and personal intent.

Human users typically: Follow natural navigation paths, click links selectively and scroll pages before submitting forms.

Bots typically: Access URLs directly without navigation, skip irrelevant pages, and interact only with targeted elements (e.g., login or comment forms).

Abnormal navigation flows can indicate automated behavior.

2.4.3 Mouse and Keyboard Dynamics

Human interaction with a web interface involves complex motor movements that are difficult to replicate precisely.

Human characteristics: Non-linear mouse movements, variable typing speed, and occasional typing errors and corrections.

Bot characteristics: No mouse movement, perfect or near-instant form completion, and synthetic or scripted input patterns.

Mouse trajectory analysis and keystroke dynamics are effective indicators of human presence.

2.4.4 Session Duration and Activity Consistency

Human sessions vary widely in duration and behavior, depending on user intent.

Human sessions: Variable session length, periods of inactivity, diverse interaction sequences.

Bot sessions: Short and repetitive, identical actions across multiple sessions and continuous operation over long periods

High behavioral consistency across sessions is a strong indicator of automation.

2.4.5 Error Patterns and Adaptive Behavior

Humans make mistakes and adapt based on feedback from the system.

Human users: Enter incorrect data occasionally, adjust behavior and learn from previous failures.

Bots: Repeat identical requests, ignore error feedback and continue attacks without adaptation.

The lack of learning behavior is a key distinguishing factor.

2.4.6 Device and Browser Fingerprint Stability

Human users typically access a website from a limited number of devices and browsers.

Human fingerprints: Relatively stable over time, consistent screen resolution and OS and natural browser feature diversity.

Bot fingerprints: Frequent changes, headless browser indicators, and limited or abnormal feature sets.

Fingerprint instability may indicate bot activity or evasion attempts.

2.4.7 Temporal Behavior Patterns

Human activity generally follows daily routines influenced by work hours, sleep, and time zones.

Human behavior: Activity peaks during daytime, reduced activity at night, and seasonal or weekly patterns.

Bot behavior: Uniform activity across all hours, no circadian rhythm and continuous operation.

Time-based analysis can improve detection accuracy.

2.5 BOT DETECTION TECHNIQUES

Detecting botnet activity in web systems is a critical security challenge due to the increasing sophistication of automated bots. Modern detection techniques aim to distinguish between legitimate human users and malicious bots by analyzing behavior, interaction patterns, and traffic characteristics. This section presents the most widely used bot detection techniques and discusses their strengths and limitations.

2.5.1 Behavior-Based Detection

Behavior-based detection focuses on identifying abnormal interaction patterns that deviate from typical human behavior.

Key features analyzed include: Request frequency and timing, navigation flow between pages, mouse movement and keystroke dynamics and Session duration and consistency.

Advantages: Does not rely solely on IP addresses, effective against distributed botnets and can detect unknown or zero-day bots.

Limitations: Requires extensive data collection, risk of false positives for power users or automated tools.

Behavior analysis is often the foundation of advanced bot detection systems.

2.5.2 CAPTCHA-Based Detection

CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) is a commonly used mechanism to differentiate humans from bots by presenting challenges that are easy for humans but difficult for automated programs.

Common CAPTCHA types: text-based CAPTCHA, Image-based CAPTCHA, behavioral CAPTCHA (e.g., Google reCAPTCHA).

Advantages: Simple to deploy, Effective against basic bots.

Limitations: Reduces user experience, vulnerable to CAPTCHA-solving services, less effective against human-assisted bots.

2.5.3 Rate Limiting and Traffic Throttling

Rate limiting restricts the number of requests that a user, IP address, or session can generate within a given time window.

Typical implementations: IP-based rate limiting, account-based rate limiting, and API request quotas.

Advantages: Easy to implement, Effective against brute force and flooding attacks.

Limitations: Ineffective against distributed botnets, can impact legitimate users behind shared Ips.

2.5.4 Browser Fingerprinting

Browser fingerprinting identifies clients based on a combination of browser and device attributes.

Common fingerprint features: User-Agent string, screen resolution, installed fonts and plugins and JavaScript and WebGL behavior.

Advantages: Difficult for bots to fully randomize, useful for detecting repeat offenders.

Fingerprinting helps detect bots that frequently change IP addresses.

2.5.5 Machine Learning-Based Detection

Machine learning (ML) techniques enable automated classification of web traffic as human or bot by learning patterns from data.

Common ML models used: Decision Trees, Random Forest, Support Vector Machines (SVM), Neural Networks.

Typical features include: Request rate, Session entropy, Interaction diversity, Behavioral consistency.

Advantages: High detection accuracy, adaptive to new attack patterns and suitable for large-scale systems.

Limitations: Requires labeled training data, computationally expensive, Risk of model bias or overfitting.

ML-based detection is increasingly used in modern web security solutions.

2.5.6 IP Reputation and Blacklisting

IP reputation and blacklisting systems assess the trustworthiness of IP addresses by analyzing historical activity and incorporating external threat intelligence feeds. These mechanisms provide a fast response and are particularly effective against known malicious sources, allowing administrators to block suspicious traffic before it reaches critical services. However, their effectiveness is limited when dealing with residential IP botnets, which often use legitimate consumer devices to mask malicious activity. In addition, reputation databases

require continuous updates to remain accurate, as attackers frequently rotate or acquire new IP addresses to evade detection.

2.5.7 Hybrid Detection Approaches

No single detection technique is sufficient to identify all types of bots, as attackers continuously adapt their methods to evade defenses. To address this challenge, modern web security systems employ hybrid detection strategies that integrate multiple approaches into a layered defense model. A typical hybrid framework begins with initial behavior analysis to identify anomalies in user interactions, followed by rate limiting to control suspicious activity. When risk levels escalate, CAPTCHA challenges are introduced to differentiate humans from automated scripts, while machine learning–based classification provides deeper analysis of traffic patterns.

This combination of techniques significantly enhances detection accuracy by leveraging the strengths of each method while compensating for their individual limitations. At the same time, the layered approach minimizes disruption for legitimate users, ensuring that friction is applied only when necessary and maintaining a balance between security and usability.

CHAPTER 3 . EXPERIMENTAL RESULTS

3.1 USING RULE SNORT

Snort is an open-source software used for network intrusion detection and prevention (IDS/IPS). It is maintained by Cisco and widely adopted in cybersecurity. Snort can analyze network traffic in real time, log packets, and detect suspicious activities.

In our test, we will use it to detect botnet intrusions.

Anomalous DNS query detected:

Botnets often use random domains (DGAs). Rules may match unusually long strings in DNS queries.

Rule snort:

```
alert udp any any -> any 53 (msg:"Suspicious long DNS query - possible DGA"; dsize:>100; sid:4000005; rev:1; metadata:policy security-ips alert;)
```

If the DNS query payload is too long (e.g., >100 bytes), it could be a DGA botnet.

Components:

alert udp any any -> any 53: Triggers on all UDP traffic from any source to port 53 (DNS server).

msg "Suspicious long DNS query - possible DGA": Message shown when the rule matches.

dsize:>100: Condition: UDP payload size greater than 100 bytes. Normal DNS queries are usually small. Queries larger than 100 bytes may indicate automatically generated domain names (DGA) used by botnets.

sid:4000005; rev:1; Rule ID and revision number.

metadata:policy security-ips alert; Metadata for Snort’s policy management.

Unusually high number of TCP connections:

```
alert tcp any any -> any any (
msg:"Possible Botnet - High connection rate";
```

```
    sid:5000001;
    rev:1;
    detection_filter:track by_src, count 50, seconds 10;
    metadata:policy security-ips alert;
)
```

Components:

track by_src: monitor traffic per source IP.

count 50: if a source sends 50 or more TCP packets…

seconds 10: …within 10 seconds, trigger the alert.

Write Snort rules: nano /usr/local/etc/rules/local.rules

Here you write all rule for snort

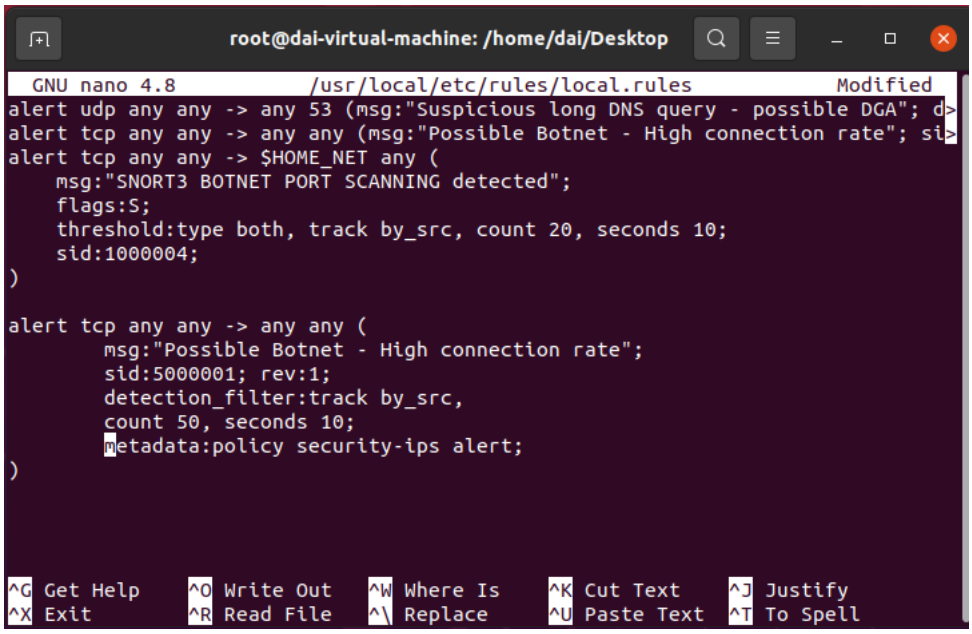

Figure 1: Folder containing rules

Run snort snort to alarm: sudo snort -c /usr/local/etc/snort/snort.lua \
-R /usr/local/etc/rules/local.rules \
-i ens33 -A alert_fast

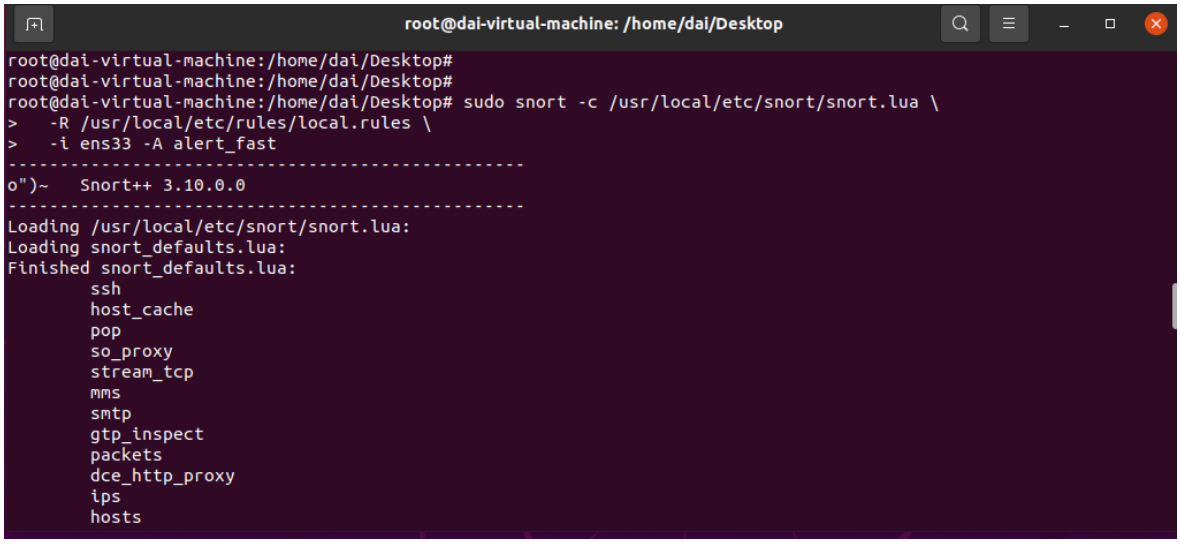

Figure 2: Run rule

Simulating botnet behavior:

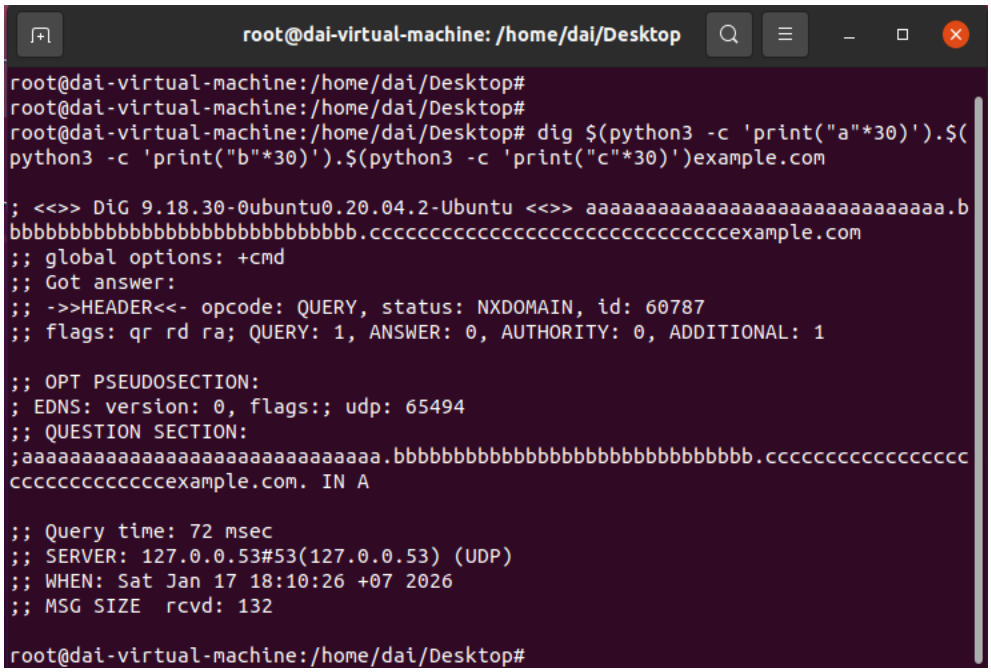

Figure 3: Botnet attack

Result: you can see log name “ Suspicious long DNS query” when botnet regulations rule, the system will alarm.

Figure 4: Alert system

3.2 MACHINE LEARNING

3.2.1 Ramdom Forest

In our lab, we primarily used the Random Forest algorithm.

The Random Forest algorithm is one of the top choices for IDS (Intrusion Detection Systems) and Botnet detection due to its ability to handle large-scale network data and its extremely high accuracy.

The Principle of "Collective Intelligence": Instead of relying on the decision of a single Decision Tree, Random Forest constructs hundreds of different trees to work together.

Random Feature Selection: When splitting a node (branching), each tree is only allowed to "see" a few randomly selected features (columns). For example, Tree #1 might focus on packet_size, while Tree #2 focuses on weight.

Voting: When a new network packet arrives, every tree in the forest makes its own prediction. If 90 trees predict "Botnet" and 10 trees predict "Normal," the final output of the system will be Botnet.

3.2.2 Machine Learning with Ramdom Forest

Load data from a file into memory and print the "shape" of the dataset.

There are 25,609 rows: Each row represents a data sample (e.g., a networkconnection session or a packet).

There are 117 columns: Each column represents a feature of that data (e.g., packet size, response time, IP address...).

1- Importing Dataset

```
data = '/content/sample_data/iot_static_data.csv'

df = pd.read_csv(data)
```

```
print("shape of dataset : " ,df.shape)
```

```
shape of dataset :  (25609, 117)
```

Figure 5: Import Data

The number of data samples for each label type is shown in the "Class" column.

BENIGN: Clean (non-harmful) network traffic.

Mirai attack: Variants of the Mirai malware attack.

Gafgyt attack...: Attack variants of the Gafgyt (Bashlite) malware.

```
df["Class"].value_counts()
```

Class	count
BENIGN	22287
mirai_udp_attack	1774
gafgyt_udp_attack	446
gafgyt_junk_attack	377
gafgyt_tcp_attack	213
gafgyt_scan_attack	200
mirai_syn_attack	101
mirai_ack_attack	96
mirai_scan_attack	86
gafgyt_combo_attack	18
mirai_udpplain_attack	11

dtype: int64

Figure 6: Label in Dataset

In the pie chart, "BENIGN" accounts for 87%, with the remainder being botnet attacks.

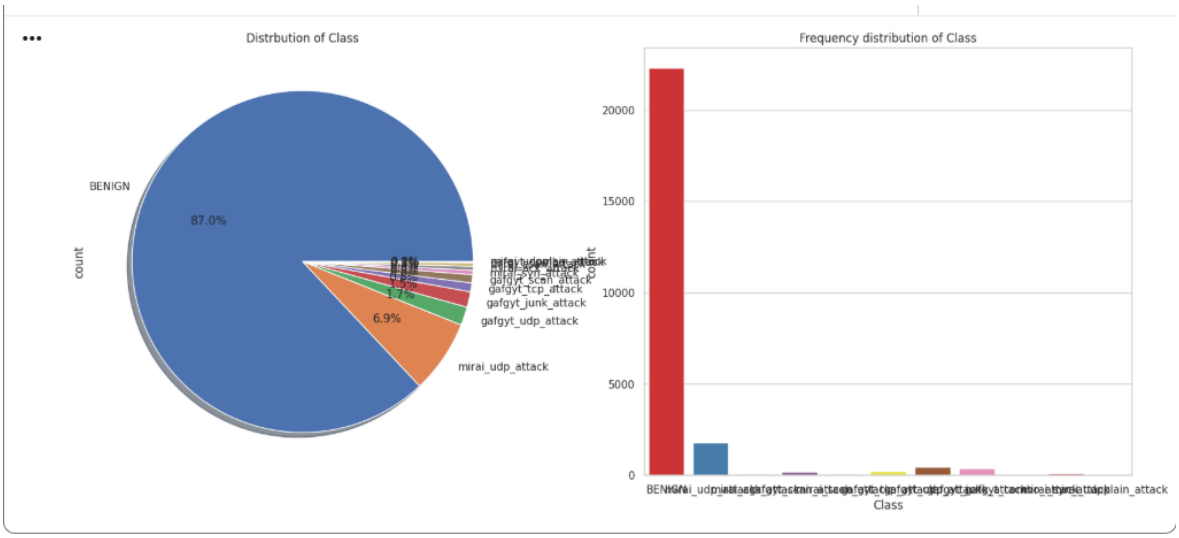

Figure 7: Data visualization

Heatmap:

On the graph, you'll see bold (red or dark blue) square blocks clustered together. These are groups of parameters that always fluctuate

together. For example, when a botnet attacks, both the "Packet weight" and "Packet size variation" spike.

Correlation with Class:

The bold cell: That parameter is the strongest "informer."

The light cell (almost white): That parameter is a "noise factor," not helpful in identifying a botnet.

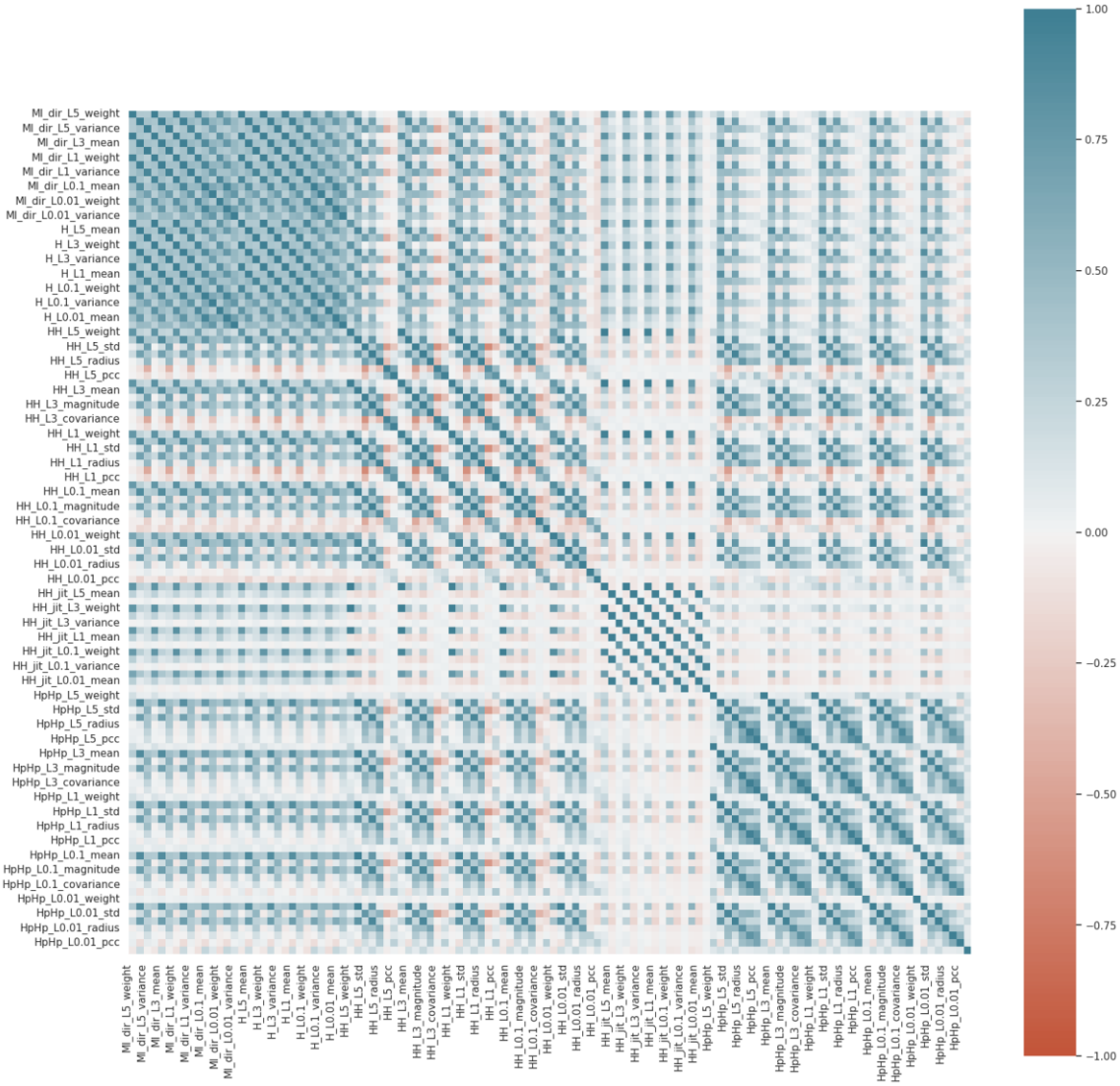

Figure 8: Heatmap

After cleaning and encrypting the data, we will move on to the most important phase: Training and Evaluation.

In this paper, we have chosen the Random Forest algorithm to identify botnets.

Feature Selection: Retain features (columns) that have an importance level of 3% or higher for predicting outcomes.

From the original 117 columns, you are left with only the 11 most important ones.

```
df_filtered.shape

(25609, 11)
```

Figure 9: Important labels

Model accuracy score : 0.9983

Apart from class 10 and class 1, the scores in the other classes were quite high.

```
Model accuracy score with selected features : 0.9983
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      6640
           1       0.20      1.00      0.33         1
           2       0.98      0.97      0.97       115
           3       1.00      1.00      1.00        60
           4       0.98      1.00      0.99        62
           5       1.00      0.99      1.00       143
           6       0.91      0.94      0.93        34
           7       0.96      1.00      0.98        26
           8       1.00      1.00      1.00        33
           9       1.00      1.00      1.00       566
          10       0.33      0.33      0.33         3

    accuracy                           1.00      7683
   macro avg       0.85      0.93      0.87      7683
weighted avg       1.00      1.00      1.00      7683
```

Figure 10: Result

Export botnet files for testing.

```
import joblib

# 1. Save the Random Forest model.
joblib.dump(clf, 'botnet_detector_model.pkl')

# 2. Save the list of 11 important filtered columns (so the computer knows which parameters to check).
joblib.dump(features.tolist(), 'selected_features.pkl')

print("Đã xuất file botnet_detector_model.pkl thành công!")

Đã xuất file botnet_detector_model.pkl thành công!
```

Figure 11: Export files

3.2.3 Testing in Kali

Place the two exported files into the Kali virtual machine.

Figure 12: File to detection botnet

Write a test file to run the two exported files.

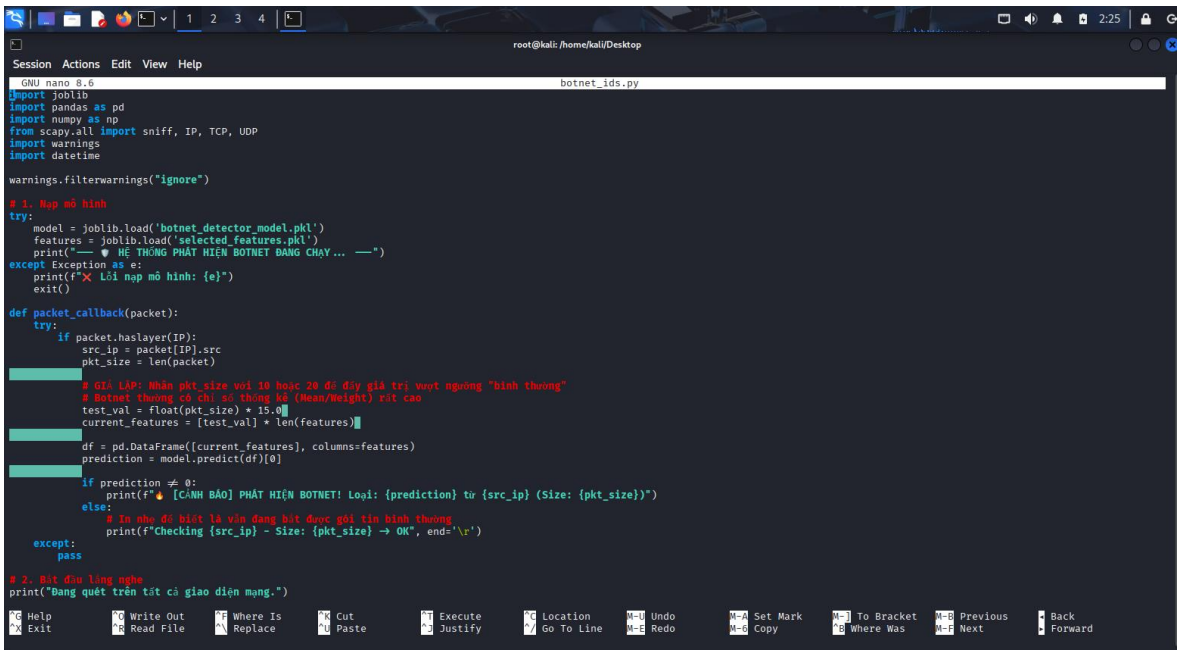

Figure 13: Code to run script

Run the monitoring script: sudo python3 botnet_ids.py

Figure 14: Run script

A simulated Ubuntu virtual machine emulates botnet behavior on Kali's machine.

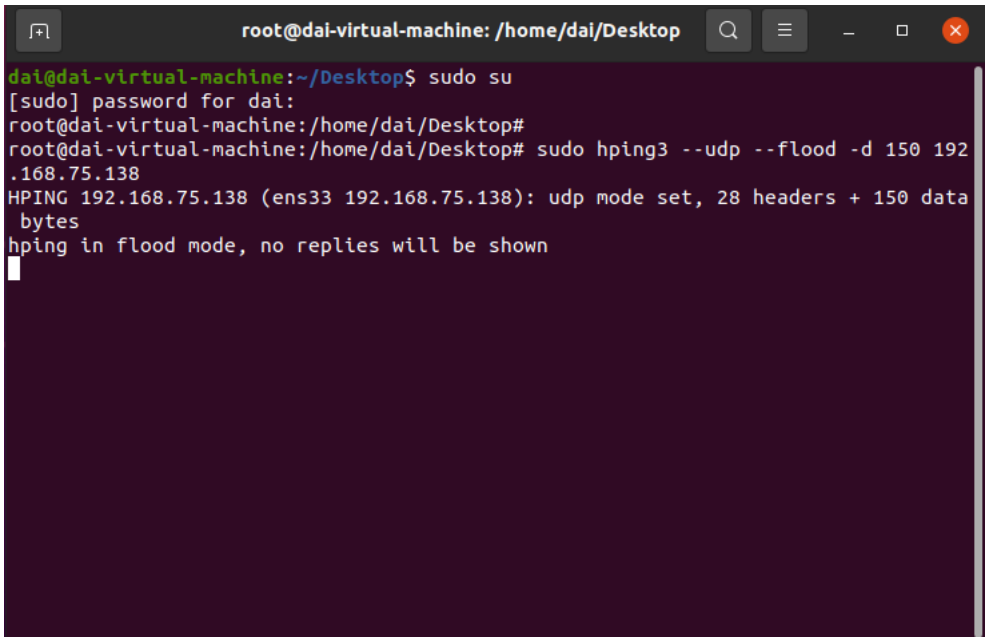

Figure 15: Attack simulator

And here are the results:

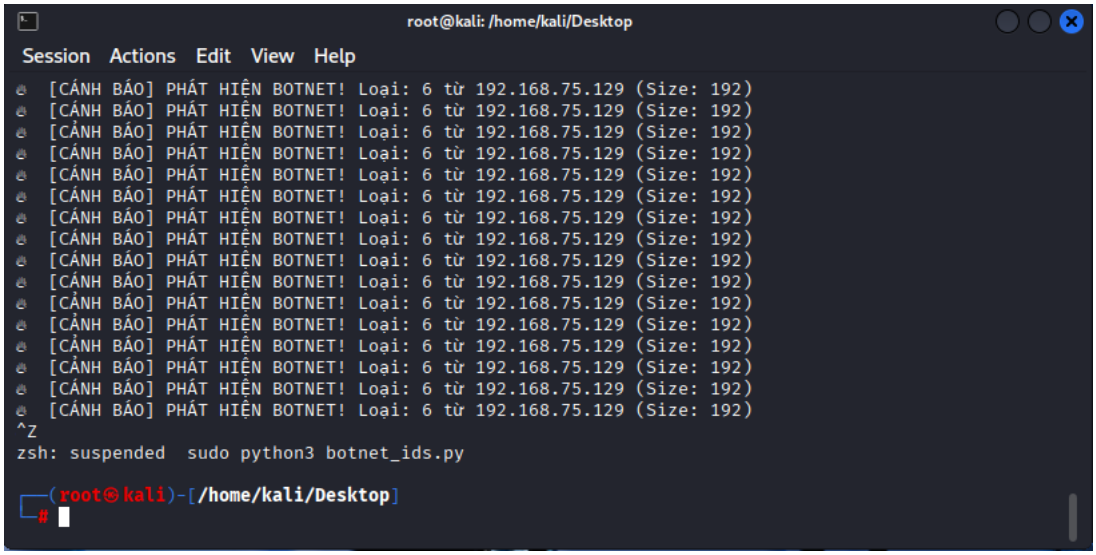

Figure 16: Detection system

CHAPTER 4 . CONCLUSION

4.1 CONCLUSION

This project focused on researching and implementing methods to detect and prevent botnet activities in web systems. Based on the analysis of botnet behaviors and detection techniques, the study demonstrated that botnets pose a serious threat to web applications through automated attacks such as credential theft, brute force login attempts, spam generation, and abnormal request flooding.

In this project, a rule-based intrusion detection approach using Snort was combined with a machine learning-based detection model to improve the accuracy of bot identification. Snort was used to monitor network traffic and detect known attack patterns at the network level, such as suspicious HTTP requests, repeated connection attempts, and abnormal traffic signatures. Meanwhile, machine learning techniques were applied at the application level to analyze user behavior and classify traffic as either human or bot based on extracted features.

Snort provided fast and reliable detection of known and rule-defined attacks, while the machine learning model enhanced detection capability by identifying previously unseen or adaptive bot behaviors. The combination of these two approaches formed a layered defense mechanism that significantly improved botnet detection compared to using a single method alone.

Overall, the project confirms that integrating network-based intrusion detection systems with intelligent behavior-based analysis is an effective strategy for protecting web systems against botnet attacks.

4.2 DEVELOPMENT

Although the proposed system achieved promising results, there are several directions for further development and improvement.

First, the Snort rule set can be expanded and continuously updated to cover a wider range of botnet behaviors, including advanced evasion techniques and encrypted traffic patterns. Integration with threat intelligence feeds could further enhance real-time detection capabilities.

Second, the machine learning component can be improved by collecting larger and more diverse datasets to reduce false positives and increase model generalization. Advanced models such as deep learning or ensemble methods could be explored to improve detection accuracy, especially for sophisticated bots that closely mimic human behavior.

Third, the current system mainly focuses on detection. Future work could extend the system to include automated response mechanisms, such as dynamic IP blocking, adaptive rate limiting, or real-time CAPTCHA enforcement based on risk scores.

Finally, the system could be deployed and evaluated in a real-world production environment to assess its scalability, performance impact, and long-term effectiveness. Additional protection mechanisms such as browser fingerprinting and API-level security controls could also be integrated to further strengthen the defense against botnet attacks.

REFERENCES

[1] A. Mailewa and K. Karunanayake, "Security threats/attacks via botnets and botnet detection/prevention techniques in computer networks: A review," ResearchGate,2019.[Online].Available:https://www.researchgate.net/publication/334126617

[2] https://www.mdpi.com/2624-800X/2/1/6

[3] https://www.kaggle.com/code/mohammednamory/iot-botnet-attack-detection-problem/notebook

Linkdemo:

https://drive.google.com/drive/folders/1cXRCiMVTFRULdHvRIxH7IGZpCZflgFk3?usp=sharing